Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 2: Model Transformations
Project Type: Code Generation
October 16, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1 Purpose of the Project **1**

1.1 Goals . . . 1

1.2 Role of the Transformations . . . 1

1.3 Refinement of Tasks 0 and 1 . . . 2

1.4 Assumptions and Conditions . . . 2

1.5 Scope . . . 3

2 Alternatives and Justification **3**

2.1 Existing Kubernetes Modelling Tools . . . 3

2.2 The Transformation Language . . . 3

2.3 The Structure of the Transformation . . . 5

2.3.1 A Lowering Before the Resolution . . . 5

2.3.2 One Project per Run, All Projects as Input . . . 5

2.3.3 Typed Extents Instead of a Global Scan . . . 5

3 Solution Design **5**

3.1 The Lowering . . . 6

3.2 Interface . . . 6

3.3 Mappings and Queries . . . 7

3.4 A Worked Example . . . 8

3.5 The Derivation Rules . . . 9

3.5.1 A Process . . . 10

3.5.2 A Dependency Edge . . . 11

3.5.3 Environment Variables and Placeholders . . . 11

3.5.4 Network Peers . . . 11

3.5.5 The Path Plan . . . 11

3.6 The Trace and Late Assignment . . . 11

3.7 The Black-Box Library . . . 12

3.8 The Estate Pass . . . 12

3.9 Stops . . . 12

3.10 Running the Transformations . . . 12

4 Tests and Evaluation **13**

4.1 Testing Strategy . . . 13

4.2 Results . . . 13

4.2.1 What the Revisions Show . . . 14

4.2.2 What the Generated Files Show . . . 14

4.2.3 Where the Evidence Is Thin . . . 14

4.2.4 Defects Found by Testing . . . 14

4.3 Limitations . . . 14

4.4 Reflection 15

A Changes to the Task 1 Metamodels **16**

A.1 The Pinned Inputs Metamodel 16

A.2 Changes to the Target Metamodel 16

A.3 Changes to the Source Metamodel 23

A.4 Grammars for the Pinned Inputs 23

B The Hand-in Archive **24**

B.1 Running the Transformations 25

1 Purpose of the Project

1.1 Goals

The project generates the Kubernetes files of a small cluster from short deployment documents. Each application's repository holds one *project document*. It says what the application needs: its processes, their memory and CPU, the ports they offer, the services they depend on, and how outside traffic reaches it. One *platform document* says what all applications share, such as the cluster's entry points, its health-check timings and its backup policies. Task 1 defined the metamodels for both in Ecore, the metamodelling language of the Eclipse Modeling Framework (EMF). The *source metamodel* describes the two documents. The *target metamodel* describes the *resolved model*: every value the platform derives for an application, and the file each value will be written to.

Task 2 builds the model-to-model transformation from source models to resolved models. It reads the source models of every project in the cluster, the platform document, and a few documents that other processes publish. Its output is the resolved model of one project, which the code generation of Task 3 reads. The transformation is where the derivation rules of the specification become executable: every value in a resolved model comes from a rule in the transformation, and no author writes those values by hand. Table 1 explains the project terms the rest of the report uses.

Term	Meaning
Process	One program of an application, which runs in its own pods.
Dependency edge	A process's statement that it calls a named port (a *surface*) of another application.
Pinned inputs	Documents the run reads besides the authored ones: the node contract (the machines and what each can hold), the images lock (each image's exact digest), the cluster-state snapshot, migration proofs, and configuration files (*Assets*).
Switchover	How a new version replaces the old one: *blue/green* (both run, then traffic moves once the new one is healthy), *rolling* or *stop-start*.
Release gate	The service that holds a blue/green switch back until every process of the application is healthy.
Reconcile unit	A group of files that Flux, the tool that applies them to the cluster, applies together and in order.
Revision	A digest (a SHA-256 fingerprint) of an application's complete resolved element, which names one release of it.
Estate	The whole set of projects, platform and pinned inputs one run reads.

Table 1: Project Terms Used in This Report

The transformation has three goals.

1. **Same decisions as production.** It derives the same values as the production implementation of the same specification, which is written in TypeScript and deploys the cluster. Both implementations are tested against the same committed result files, called *oracles*.
2. **No silent defaults.** It stops with a message when it cannot derive a value. A resolved model with a missing or invented value would generate files that look valid and deploy the wrong thing.
3. **Nothing left to the templates.** Its output is complete enough that the templates of Task 3 only write values out. A template that had to choose a path, a port or a name would hide a derivation from every check.

1.2 Role of the Transformations

Figure 1 shows the pipeline. Xtext turns each document into a source model, and the OCL constraints of Task 1 reject invalid models. Three QVT-Operational (QVTo) transformations then run in a chain.

1. The *lowering* copies what an author declared once for a whole project or application onto each process. Its output, the *Effective Intent*, uses the source metamodel too. It was part of the Task 1 implementation but not of the Task 1 report, so Section 3.1 describes it.
2. The *resolution* reads the lowered models and the pinned inputs and writes one resolved model. It is the main subject of this report.

3. The *estate pass* runs once over the resolved models of all projects generated together. It adds the files that belong to no single project (Section 3.8).

A small Java exporter writes the resolved dependency edges as JSON, which is compared with the oracle. The Acceleo templates of Task 3 read the same resolved models and write the Kubernetes files.

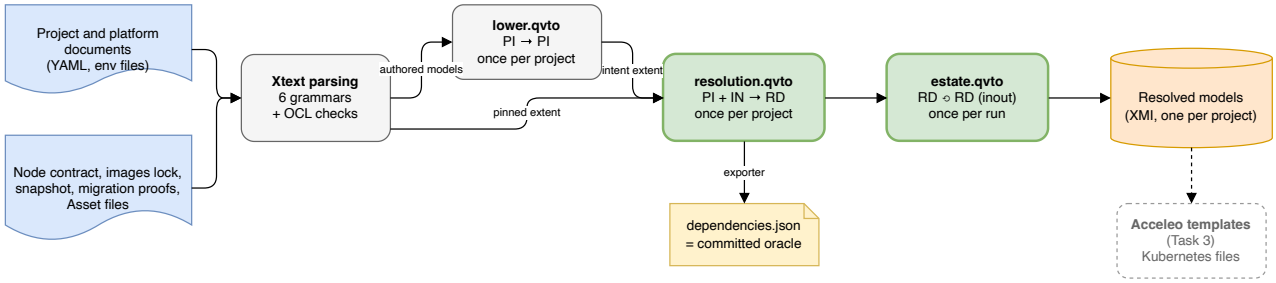


Figure 1: The Pipeline, With the Two Transformations Task 2 Adds in Bold Green

1.3 Refinement of Tasks 0 and 1

Writing the transformation refined the earlier tasks in five ways.

- **The pinned inputs became models.** Task 0 listed the node contract, the images lock and the snapshot as inputs the generator is given, and Task 1 modelled only the two authored documents, stating that only those have metamodels. The resolution needs the pinned inputs as models, so a third metamodel now describes them, with an Xtext grammar per document (Appendix A.1). It is a separate package because no project author writes these documents.
- **A lowered stage.** Task 1 named three stages: authored, resolved and generated text. The lowering adds a fourth between the first two, so that each rule of the resolution reads one level of values instead of three (Section 2.3.1).
- **One resolved model per project.** Task 0 ran each generator once over all projects, and Task 1 planned one resolved model for the whole cluster, because edges, routes and reverse dependencies need a cluster-wide view. That view is kept: every run still reads every project. Only the output is per project, because the production implementation writes one resolved document per project and the oracles are per project (Section 2.3.2). Task 1's per-application *projection* is still the `ResolvedApplication` inside that model.
- **An estate pass.** One generated job, which writes every project's secret-store policies, is named after a digest over all of them. No single project's resolution can compute that name, so a small second transformation over all resolved models plans the job.
- **The target metamodel grew.** Task 1's target metamodel had a worked instance only for `minimal`. The other examples need more: migrations, backup identities, API access, sidecar containers, and twelve more kinds of generated file. Appendix A lists every change.

1.4 Assumptions and Conditions

- **The input is valid.** The OCL constraints reject an invalid source model before the lowering runs, over the whole set of documents of the run. A stop in the transformation is a safety net: it fires only for a fact the constraints do not check yet (Section 3.9).
- **One cluster per run.** Each run reads one platform document and one node contract, and both name one cluster. The concepts, the examples and the oracles all come from one existing cluster.
- **The production implementation is the reference.** Where the specification leaves a choice, the transformation makes the same choice as the production implementation, so that both produce the same oracles.
- **No live cluster access.** The transformation reads only the documents it is given. Facts that only a running cluster knows, such as which machine holds a volume, arrive through the snapshot, which another process captures.
- **Runs without Eclipse.** The build runs the transformations with QVTo's standalone executor on every change. Eclipse run configurations also run them (Appendix B).

More Than One Cluster. The single cluster is a choice of scope, not a limit of the structure. Everything specific to a cluster is already an input of the run: the platform document, the node contract and the snapshot each name their cluster, and a project's environment files are a shared base plus an overlay per cluster, which the resolution selects by the platform's cluster name (Section 3.5.3). A second cluster would therefore be a second run with that cluster's three documents. Two things would be new, and neither is built or tested:

- a project document would have to say which clusters it runs in;
- an edge to a service in another cluster would resolve through the platform's list of external providers, as the edge from `auth` to a mail server outside the cluster already does (Section 3.5.2), but its network policy would have to admit an address range instead of a pod.

1.5 Scope

Table 2 sets the scope of Task 2.

In scope	Out of scope
The resolution of eight examples: `minimal`, `auth`, `data`, `delivery`, `edge`, `knowledge-platform`, `observability` and `secrets`.	The `knowledge` example, which models a project being retired (Section 4.3).
Every derivation these examples use, from dependency edges and placement to backups, migrations, network peers and the path of every generated file.	The text of the generated files, which is Task 3.
The lowering's extensions, the estate pass, and the pinned-inputs metamodel and grammars.	Applying the files to a cluster, and a second cluster.

Table 2: Scope of Task 2

2 Alternatives and Justification

This section answers three questions. Could existing Kubernetes tools replace the transformation (Section 2.1)? Which language should it be written in (Section 2.2)? How should it be structured (Section 2.3)? It gives the answers in general terms; Section 3 gives the detail.

2.1 Existing Kubernetes Modelling Tools

Three open-source tools that model Kubernetes applications were reviewed, two of them graphical. For each, the questions were: can it produce the resolved model or the files from what an author declares? If not, can it take part in the pipeline? Table 3 gives the answers.

None of the three derives values from what an author declares. KubeOrch comes closest: its automatic wiring resolves a link between two services to an address and a port, as the resolution does for a dependency edge. But it has no platform document, node contract or images lock, so it cannot derive placement, image digests, backups or network peers, and its result is not a model that a constraint or an oracle can check. The tools can therefore sit beside the transformation, not replace it. Two integrations follow from the design:

- the resolved dependency edges and network peers are exactly the links a KubeDiagrams view of the Task 3 files would draw, so the drawing is a cheap visual cross-check of `dependencies.json`;
- a graphical editor in KubeOrch's style could write project documents, because its models and the Xtext grammar's would be instances of the same Ecore metamodel.

Neither is built, because neither changes what the transformation must derive.

2.2 The Transformation Language

Four languages were candidates: ATL, QVT-Relations, QVT-Operational (QVTo), and Java, by hand or with a small rule library such as SiTra. QVT-Relations was dropped first. Its strengths, automatic traces and synchronisation in both directions, buy nothing for a one-way derivation, and in published experiments its engine

Tool	What it does	Why it cannot replace the resolution	Where it could help
Kube-Diagrams [1]	Draws a diagram from Kubernetes files or a live cluster, with a configurable mapping from resource kinds to icons and edges.	It goes the other way, from files to a picture, and derives nothing.	A view of the generated files, showing the links a project document produces.
KubeOrch [2]	A drag-and-drop editor of services; its back end wires their ports, fills in templates, writes Kubernetes files and deploys them.	A stored graph without metamodel or constraints; rules in Go code; deploys directly, not through Flux.	A graphical notation: its graph maps onto a project document's applications and dependency edges.
kubert [3]	A command-line tool that switches cluster contexts and runs one command on several clusters.	An operations tool; it models and generates nothing.	Applying files to several clusters, which is out of scope.

Table 3: Existing Kubernetes Modelling Tools Against the Problem

ran out of heap space on larger models [5]. Table 4 compares the other three against what this transformation has to do.

Requirement	ATL	QVTo	Java
Derive a value from the whole cluster, not from one source element	helpers outside the rules	queries, in OCL	methods
Create elements in a fixed order	order of matched rules and bindings unspecified	explicit, by call order	explicit
Refer to a target element another rule created	implicit, for matched and lazy rules	explicit `resolveone` through the trace	a trace kept by hand
Set a value once the rest of the element exists	imperative `do` section	`end` section	any order
Change a model in place (the estate pass)	refining mode	`inout` extent	any

Table 4: Transformation Languages Against the Requirements of the Resolution

The deciding property is the shape of the derivations. Most values in a resolved model do not come from one source element: the eligible machines of a process depend on the node contract, a route's precedence on the other routes, and the ingress peers of a process on every process in the cluster that depends on it. ATL is strongest when one source element maps to one target element, through matched rules whose order the engine chooses. Here most of the work would move into helpers and called rules, which is imperative ATL without the matching, and several values need the rest of the model to exist first. QVTo makes both visible: the caller decides the order, and every reference from one created element to another goes through the trace explicitly. Its expressions are OCL, which the constraints of Task 1 already use. A Java transformation would be the easiest to debug, but it keeps the trace by hand and hides the rules in general-purpose code, which practitioners themselves find hard to maintain for model management [4].

QVTo was therefore chosen. It is also the second most used transformation language after ATL, and its users name its maturity and its standard as the reasons [4]. Speed did not decide the choice: QVTo ran more than five times slower than ATL in published experiments [5], which is acceptable for a cluster of about ten projects. The one operation QVTo cannot express, a SHA-256 hash, comes from a Java black-box library, the mechanism QVT defines for calling external code (Section 3.7).

2.3 The Structure of the Transformation

Table 5 lists the structural questions, the alternatives considered, and the choice made.

Question	Alternatives	Choice and reason
Where values are derived	In the Acceleo templates of Task 3; or in a model-to-model step before them	A model-to-model step. Every derived value is then in a model that a test can compare, and the templates only write values out.
One transformation or a chain	One from the authored model; or a lowering, the resolution and an estate pass	A chain of three. The resolution reads one level of values instead of three, and the estate pass computes the one value that needs every project's output.
How much to resolve per run	The whole cluster in one model; or one project per run, reading all	One project per run, reading all. It matches the production implementation and its oracles, and keeps each model small.
How other documents are found	`allInstances()` over everything loaded; or typed extents, read once into properties	Extents. Every query sees exactly the documents of the run.
What to do with an underivable value	Write a default; write a diagnostic model; stop with a message	Stop. A default would produce files that deploy the wrong thing.

Table 5: Structural Alternatives and the Choices Made

2.3.1 A Lowering Before the Resolution

An author may declare a secret, a dependency or a startup budget once for a whole project, once for an application, or on a single process. The lowering merges these levels onto each process, so that each process carries everything that applies to it. If the resolution read the authored model instead, every rule that reads a process would have to merge three levels again, and could merge them differently in different rules. With a lowering, each rule reads one place. A preparation step before the main transformation is also the QVT specification's own example of composing transformations [6, Section 8.1.13]; here the Java pipeline chains the three instead of QVTo's `access transformation`, so each can be run and tested on its own.

2.3.2 One Project per Run, All Projects as Input

The resolution runs once per project, with the other projects as inputs: `auth` reads `data` to find its database's address, and `data` reads `auth` to find who may reach it. One model for the whole cluster would also work, since QVTo can create several root objects in one run, but the production implementation and its oracles are per project, and a project's model is then what its repository publishes. The cost is the one value that needs every project's output, which the estate pass computes.

2.3.3 Typed Extents Instead of a Global Scan

A rule could find another document with OCL's `allInstances()`, as some cross-document constraints of Task 1 do. That costs a scan of everything loaded on each evaluation, and gives a result that depends on which files happened to be loaded together. The transformation avoids both: each input is selected once, by type, from two extents that hold exactly the documents of the run, and every rule navigates from there (Section 3.2). The constraints still use `allInstances()`, guarded so that they hold over a file read alone. The pipeline therefore checks each file alone and then the whole set of a run together, and the platform document is always in that set, so a guarded constraint never skips in a run. Moving those constraints onto one root object that holds the set, so that they too navigate instead of scan, is decided but not yet built (Section 4.3).

3 Solution Design

The resolution is one QVTo transformation, `resolution.qvto`, of about 1700 lines: 8 mapping operations and 140 queries. The estate pass, `estate.qvto`, adds about 130 lines, and the lowering, `lower.qvto`, about

340. This section first classifies the three and describes the lowering (Section 3.1), then the resolution's interface (Section 3.2), the division into mappings and queries (Section 3.3), a worked example (Section 3.4), the derivation rules (Section 3.5), the trace and late assignment (Section 3.6), the black-box library (Section 3.7), the estate pass (Section 3.8), the stops (Section 3.9), and how the transformations are run (Section 3.10).

Figure 2 shows which metamodels the resolution and the estate pass read and write; each transformation names them in `modeltype` declarations, by their namespace URI. The three transformations differ in kind, and each kind was chosen for its job. All three are one-way:

- the *lowering* is endogenous and horizontal: it reads and writes the source metamodel, at the same level of abstraction, into a new model (out-place), one project in, one out;
- the *resolution* is exogenous and vertical, a refinement that adds platform detail: it reads the source and pinned-inputs metamodels and writes the target metamodel, many models in (N), one out (1);
- the *estate pass* is endogenous and in-place: it updates the resolved models of a run through an `inout` extent (M to N).

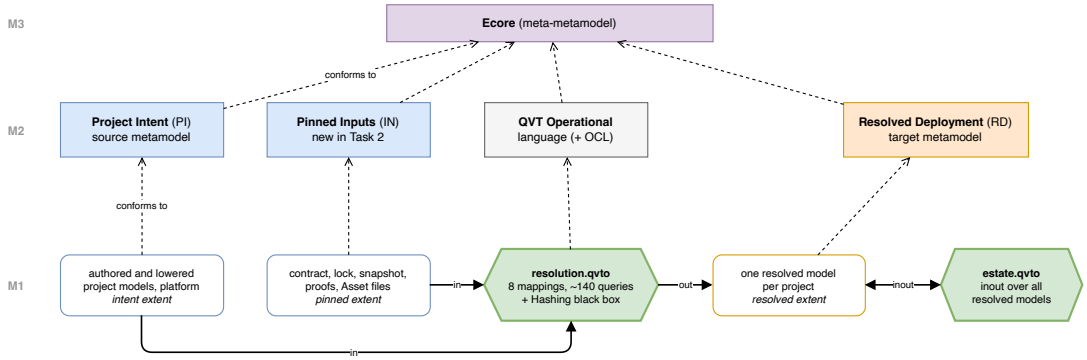


Figure 2: The Resolution and the Estate Pass in the Metamodel-Based Transformation Pattern

3.1 The Lowering

The lowering, `lower.qvto`, turns each authored `Project` into an `EffectiveProject`, in which every process carries all the shared declarations that apply to it and the project and applications carry none. It was part of the Task 1 implementation; Task 2 extended it with API access and engine grants. It has 37 small mappings, nearly all copies, and three merge rules:

- a list (secrets, dependency edges, Assets, environment entries) is the union of the three levels, keyed by each item's own identity, and the lowest level that declares an item wins;
- a single value (a startup budget, a cutover) comes from the lowest level that declares it;
- memory and CPU come only from the process itself.

Two details are specific to QVTo. A mapping returns its cached result when it is called again on the same source, so a declaration shared by three processes would become one object that the last process takes into its containment. Every shared copy is therefore a mapping with the process as a parameter (`copyFor(self)`), which keys the trace on the pair. And a migration is one of three subclasses, so its copy is a `disjuncts` mapping that selects the subclass at run time. The lowering has no stops: the constraints run before it, so every input it sees is valid.

3.2 Interface

Example 1 shows the signature. The resolution reads two *extents*, the sets of models a transformation is given, of two metamodels, and writes one extent of a third.

```
transformation resolution(in intent : PI, in pinned : IN, out resolved : RD)
        access library Hashing;
configuration property project : String;     -- the project this run resolves
property estate : Platform = intent.rootObjects()[Platform]->any(true);
```

```
property projects : Sequence(EffectiveProject) =
        intent.rootObjects()[EffectiveProject]->sortedBy(p | p.project)->asSequence();
property lock : ImagesLock = pinned.rootObjects()[ImagesLock]->any(true);
-- and likewise contract, snapshot, proofs and assetFiles
main() {
        projects->select(p | p.project = project)->any(true).map deployment();
}
```

Example 1: The Signature and Inputs of the Resolution, Abridged

The `intent` extent holds, for every project of the run, its authored model and its lowered model (`EffectiveProject`), and the platform document. The authored models stay in the extent only because the platform document refers into them: an entry point names the proxy application that serves it, as an Ecore reference. Every rule reads the lowered models. The `pinned` extent holds the node contract, the images lock, the snapshot, each project's migration proof, and one `AssetFile` per configuration file. Two configuration properties give what is not derived: the project to resolve, and the integrity digest of the schema package, which the resolved model records as provenance.

The properties select each input once, by type, from the roots of its extent. This is the only way the transformation reaches other documents: every rule navigates from one of the seven properties, and no rule calls `allInstances()`. A query over "every process in the cluster" is `projects.applications.processes`, whose extent is exactly the run's documents. The projects are sorted by name, so that "the first provider" of a service is the same in every run and in both implementations.

3.3 Mappings and Queries

QVTo offers two ways to create a target object. A *mapping operation* records its source and result in the trace, the record of which target object was made from which source. Other rules find the result later with `resolveone`, and calling the mapping twice on the same source returns the same result. *Object creation* (an `object` expression) inside a query makes an object that no other rule needs to find. The transformation uses a mapping exactly where another rule refers to the result, and object creation everywhere else. Each target type has exactly one mapping, so the type alone identifies the result: `p.resolveone(ResolvedProcess)` needs no `resolveIn`. No mapping has a `when` guard or inherits from another: the caller selects what to map with `->select`, which keeps the choice in one visible place. Table 6 lists the eight mappings.

Mapping	What it creates, and why it is a mapping
`EffectiveProject::deployment`	The `ResolvedDeployment`. Its `end` section sets the values that need the complete model.
`EffectiveProject::unit`	The project's reconcile unit, the group Flux applies together. Each application refers to it.
`EffectiveProject::provenance`	The digest of every input and the hash over them; its `end` section takes the hash.
`EffectiveApplication::application`	A `ResolvedApplication`. Generated files and the revision refer to it.
`EffectiveApplication::gate`	The release gate's inputs. The application and its configuration file refer to them.
`EffectiveApplication::migrationOf`	The database migration of an application that manages its schema. Its network policy refers to it.
`Process::process`	A `ResolvedProcess`. Gate members, routes, scrapes, policies and generated files refer to it.
`Exposure::exposure`	A `ResolvedExposure`. The generated route file refers to it.

Table 6: The Mapping Operations of the Resolution

The queries are side-effect free, and many are attached to a model type, such as `Grant::resolvedFor`. They are of four kinds:

- *value queries* derive one value, such as a deadline or an address;
- *object queries* build a contained object, such as a probe or a backup plan;

- *spelling queries* turn an enumeration literal of the source metamodel into the string the target records, for example `AlertClass::businessHours` into `business-hours`;
- *stop queries*, such as `unlocked`, raise a fatal assertion instead of returning a value (Section 3.9).

The one QVTo *helper*, an operation with side effects, is in the estate pass, which adds files to a model it updates in place.

The mappings run in a fixed order. The `deployment` mapping creates the reconcile units first, then the applications, the provenance and the path plan. Each application maps its processes before its exposures, gate and migration, because those refer to the processes through the trace.

3.4 A Worked Example

The `minimal` example declares one application, `notes`, with one process, `notes-api`, in about 35 lines of YAML without comments. Table 7 follows some of its declarations through the resolution to the values they derive, with the platform document and the pinned inputs that also take part.

Declared	Rule	Derived in the resolved model
`image: notes-api`	`locked`	`ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e...`, user and group 1000, from the images lock
`startupBudget: 20s`	`deadlineOf`, startup probe	deadline `60s` (three budgets); a startup probe on the liveness target every `5s`, 4 failures allowed
`cutover: continuous`	`switchoverOf`, `gate`	switchover `blue-green`; a release gate with one member, deadline `60s`, the platform's analysis cadence, and error-rate and latency checks because the `node` runtime exports telemetry
`memory: 256Mi`, `cpu: 50m`	`eligible`	all seven machines of the node contract, each tested for twice the request, since old and new run side by side during a blue/green switch
`audience: anonymous`	`tierFor`, `exposure`	entry point `public-frankfurt` with TLS and ACME certificates, proxy `traefik-public` in `edge-system`
route `/`, `prefix`	`precedenceOf`, `middlewareOf`	precedence 1; security headers with the `strict` content policy
`runtime: node`	`profileOf`	`OTEL_SERVICE_NAME`, `PORT=8080`, `DEPLOYMENT_ENVIRONMENT`, and the collector's address, found in the `observability` project
`scrape` on `http`	`ingressOf`, `monitorOf`	ingress from `traefik-public` and `prometheus` on port 8080; a pod monitor, because a blue/green process is scraped through its pods
(nothing)	path plan	12 generated-file entries, among them `apps/notes/namespace.yaml` and `estate/edge/public-frankfurt/notes-public.yaml`

Table 7: Declarations of `minimal`, Followed Through the Resolution

Four of these values need another document of the run: the image needs the lock, the eligible machines the node contract, the proxy the `edge` project, and the collector's address the `observability` project. Example 2 shows part of the source model and the part of the resolved model the transformation writes for it.

```
- name: notes-api
  image: notes-api
  placement: { memory: 256Mi, cpu: 50m }
  startupBudget: 20s
  cutover: continuous
--------------------------------------------------------------------------
<processes name="notes-api" deadline="60s" switchover="blue-green"
```

```
    image="ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e5a4b..."
    uid="1000" gid="1000" memory="256Mi" cpu="50m">
  <startup period="5s" timeout="5s" failures="4"> ... </startup>
  <placement> <eligibleNodes>enschede-t1000-1</eligibleNodes> ... </placement>
</processes>
```

Example 2: Part of the Process `notes-api`, as Authored (Top, Condensed) and as Resolved, in XMI (Bottom, Abridged)

3.5 The Derivation Rules

Table 8 lists the rules by the target element they produce. Each implements a derivation of the specification; where the specification leaves a choice, it follows the production implementation.

Target value	Rule	Derivation
Application		
reconcile unit and order	`unit`, `afterOf`	One unit per project; it follows the units of the projects its edges reach, and the secrets unit if it holds a secret.
secret store	`synced`, `storeEndpoint`	The secret store's address, where a secret, backup or migration needs it.
migration	`migrationOf`, `datastoreOf`	Runner image (lock), tested schema version (proof), the database's address (the application's edge to it), an owner credential, and egress.
release gate	`gate`, `switchesBlueGreen`	One member per blue/green process with readiness; the slowest member's deadline; the platform's cadence; checks where telemetry is exported.
Process		
image, user, group	`locked`	The lock entry for the image name.
switchover	`switchoverOf`	`stop-start` for an interrupted process, `rolling` for the platform's delivery tools, `blue-green` otherwise.
deadline	`deadlineOf`	Three startup budgets, rounded down; the budget, rounded up, for a process that prepares a release; 600 seconds without a budget.
probes	`Probe::cadenced`	The declared target with the platform's period, timeout and failures. The startup probe polls the liveness target every 5 seconds, for as many attempts as the budget allows.
eligible machines	`eligible`	Every machine whose memory and CPU fit the process with its sidecars, twice for a continuous process, and that has every declared architecture, site, capability, GPU class and disk medium.
environment	`environmentOf`, `entryFor`	The base env file overlaid by the cluster's own, placeholders replaced, then the runtime profile's variables; sorted by name.
secrets	`Grant::resolvedFor`	A key-value grant keeps its path and keys and gets the Kubernetes Secret it is synced to. A transit or database grant becomes the Vault paths its policy covers.
volumes and backups	`backupFor`, `backupEgress`	For a recoverable or irreplaceable volume: the platform's schedule and retention, the backup image for the database engine, a backup identity and claim, where the backup connects, its credential, and its off-cluster copy.
configuration files	`Asset::resolvedFor`	The file's text, under a name of the process, the file and the first ten hex digits of the text's digest, so a changed file gets a new name.

Target value	Rule	Derivation
dependency edges	`edgeOf`	The first process in the cluster whose application and port the edge names, with its address and network peer; otherwise the register of external providers.
Exposure		
entry point	`tierFor`	The first entry point that carries the audience.
route precedence	`rankOf`, `precedenceOf`	Exact routes before prefix routes, longer prefixes before shorter.
middleware	`middlewareOf`	Authentication for an authenticated route, security headers, then a redirect if declared.
Network policy, per process		
ingress	`ingressOf`	The proxy of each route's entry point, the metrics collector if scraped, every process with an edge to one of its ports, its own backup identity, the migrations of the databases it holds, and the delivery events receiver. Each peer once per rule that admits it.
egress	`egressOf`	The cluster's DNS, and the secret store if the process holds a secret.
Deployment		
provenance	`provenance`	One digest per input, and a hash over all of them.
revision	`end` of `deployment`	The digest of each application's complete element (Section 3.7).
generated files	`projectFiles`, `applicationFiles`, `edgeFiles`, `vaultFiles`	The path of every file, with the elements it is generated from (Section 3.5.5).

Table 8: Derivation Rules, by Target Element

3.5.1 A Process

Example 3 shows the start of the process mapping. The `init` section looks up the lock entry once, so the assignments can use it. Each assignment calls a query of Table 8. An optional feature is assigned only when its condition holds: assigning an unset value would mark the feature as set, and the model would then state a value that nothing derived.

```
mapping PI::Process::process(in app : EffectiveApplication, in prj : EffectiveProject)
                : ResolvedProcess {
        init { var own : LockedImage := locked(self.image); }
        name := self.name;
        image := own.repository + '@' + own.digest;
        if self.lifecycle = Lifecycle::application
                then switchover := switchoverOf(app, self) endif;
        deadline := inSeconds(deadlineOf(self));
        secrets := self.secrets->collect(g | g.resolvedFor(self));
        placement := object ResolvedPlacement {
                eligibleNodes := contract.nodes->select(n | eligible(self, n))->collect(n
    | n.name);
        };
        dependencies := self.dependsOn->collect(e | edgeOf(e));
        environment := environmentOf(self, prj)->sortedBy(e | e.name);
}
```

Example 3: The Process Mapping, Abridged

3.5.2 A Dependency Edge

A dependency edge names an application and one of its surfaces, for example `platform-postgres` and `postgres`. The query `edgeOf` finds the first process in the cluster that belongs to that application and provides that surface. The resolved edge records the process's in-cluster address and the network peer the consumer may reach. If no process provides the surface, the platform's list of external providers is tried; this is how `auth` reaches the mail server outside the cluster. If neither has it, the run stops.

While a project still runs at its old location, as the platform's handover ledger records, its address and peer use that location's namespace and name, so a consumer can depend on it before it moves.

3.5.3 Environment Variables and Placeholders

A process's env files are a base file and, optionally, one overlay per cluster; the resolution keeps the base entries the overlay for the platform's cluster does not replace. An entry may contain a placeholder, such as `DB_HOST=${dependency:platform-postgres.host}`. The query `Placeholder::entryFor` handles four kinds. A *secret* placeholder becomes a reference to a key of a secret the process holds, so the value never appears in a file. A *dependency* placeholder takes the host or the port of the process's own edge, from `edgeOf`. An *exposure* placeholder takes the URL, host or scheme of an exposure anywhere in the cluster. An *identity* placeholder takes the process's name, namespace or Vault role. Text after a placeholder is kept, as in `AUTH_LOGIN_URL=${exposure:auth.public#url}/login`. The runtime profile then adds its own variables; an author who writes one of them in an env file stops the run, because one name would have two values.

3.5.4 Network Peers

The ingress of a process is a union of six sources (Table 8), and most of them look at other projects: the consumers whose edges name it, the migrations of the applications whose database it holds, and the proxies and metrics collector that the platform names. Each source yields tuples of rule, namespace, process and port; the union is made distinct with `asOrderedSet()` before the tuples become `PolicyPeer` objects. Building plain tuples first matters: two objects are never equal in OCL, so a union of objects would keep a peer that two routes admit twice.

3.5.5 The Path Plan

The *path plan* assigns each generated file its path, as a `Deliverable` subclass that refers to the elements it is generated from. The project gets a namespace file, an index, a default-deny network policy, and the secret operator's connection where an application syncs secrets. Each application gets a directory with one file per kind of resource: workloads, Canaries for blue/green processes and Services for the others, service accounts, RBAC, volume claims, the configuration map (which also carries the release gate's inputs as JSON), backups, disruption budgets for processes with more than one replica, the migration, the monitor, synced secrets, and a network-policy file with one policy per process. A file that would be empty is not planned. Each index lists the files of its directory that Flux applies; the network policies are generated but not yet applied, so no index lists them. Each exposure gets a route file under its entry point, and each identity that holds a secret a Vault policy and role. Because the paths are in the model, two files that claim one path can be found before any text is written, and the templates of Task 3 decide no path.

3.6 The Trace and Late Assignment

Every reference from one created element to another goes through the trace. A gate member, a route, a scrape and every per-process file refer to a `ResolvedProcess`, found with `p.resolveone(ResolvedProcess)` on the source process. This works because the application mapping maps its processes first. Four values can only be computed once the rest of the model exists, and they are set in `end` sections, which run once the mapping's body has finished:

- each application's *revision*, the digest of its complete element, which names the release and which every generated Canary carries;
- each migration file's tag, the first digits of that revision;

- the provenance's render hash, which covers every input digest;
- the order between reconcile units, which refers to units the same mapping created.

3.7 The Black-Box Library

A digest is a SHA-256 hash over a canonical text. OCL has no operation that hashes bytes, so the Java class `Hashing` provides three contextual operations: `digestOf` walks any model element reflectively, writes canonical JSON and hashes it; `textDigestOf` hashes a file's text as the production implementation does; and `rawDigestOf` hashes a string as it stands. The transformations call no other Java code.

The revision uses the third. Generated files carry it, so it must equal the production implementation's revision, which is the digest of the application's element in that implementation's JSON form, a shape the target metamodel does not have. The query `elementJson` therefore writes that element as text, in OCL, from the source and the target model together: keys sorted, absent values left out, every value spelled as the production implementation spells it. The black box only hashes the result. This keeps every decision about what the revision covers inside the transformation, and it makes the revision a strong test (Section 4.2).

Two details of the QVTo implementation shaped the library. An operation that takes a plain `EObject` is not found on a model element, because a model type does not conform to the Ecore type in the compiler, so each call casts first: `p.oclAsType(ECORE::EObject).digestOf()`. And the unit cannot be named after the Java package, because `resolve` is a QVTo keyword, so it is registered as `dev.jorisjonkers.deploykit.emf.hashing`, by the runner outside Eclipse and by `plugin.xml` inside it.

3.8 The Estate Pass

One generated job writes every project's Vault documents into the secret store, and it is named after a digest over all of them, so that changing any document starts a new job. No single resolution can compute that name. The estate pass, `estate(inout rendered : RD)`, runs once over the resolved models of all projects generated together. It collects every `VaultPolicyFile` and `VaultRoleFile`, writes each as canonical JSON in OCL, sorts them by file name, hashes the whole with `rawDigestOf`, and adds the job's four files and their index to the first model. It also adds the route that carries the delivery tool's events to the platform's reporter, which likewise belongs to no project. It stops if two models carry different jobs, because one run has one platform. Its source and target are the same metamodel and it changes the models in place, which QVTo expresses with an `inout` extent.

3.9 Stops

Fifteen fatal assertions in the resolution and one in the estate pass stop the run when a value cannot be derived. Thirteen live in stop queries, which a rule calls instead of returning an invalid value: `locked` returns the lock entry for an image, or calls `unlocked`, whose `assert fatal` ends the run with `E_UNLOCKED_IMAGE`; two check the release gate's preconditions at the start of the `gate` mapping, and the estate pass's one stops when two projects carry different policy jobs.

The stops cover an image missing from the lock; an edge that no process or provider serves; a placeholder that names nothing or a secret the process does not hold; a runtime-profile variable written by an author; an Asset whose file is missing or contains a placeholder; a backed-up volume without a schedule or a database engine; a migration that reaches no database; a blue/green application without readiness, release gate or delivery policy; a grant without a secret store; and a quantity in a unit the model does not read. Where the specification defines an error code, the message contains it. An audience that no entry point carries has no stop of its own, because a constraint (`E_NO_TIER_FOR_AUDIENCE`) already refuses it before the run.

3.10 Running the Transformations

Three small Java classes, `Lowering`, `Resolution` and `Estate`, run the transformations through QVTo's standalone `TransformationExecutor`: they register the black box, set the configuration properties, pass the extents, and return the result or fail with the executor's diagnostic. The pipeline builds the extents from the parsed and checked documents and writes every example's models under `target/parity/<example>/`; Appendix B.1 gives the steps.

4 Tests and Evaluation

4.1 Testing Strategy

The transformations are tested by running them on the eight resolved examples and comparing what they write with expected files. Most expected files were produced by the production implementation from the same documents and reviewed before they were committed; the production implementation is tested against the same files. Six kinds of test each answer a different question, and the build runs all of them on every change.

1. **Lowered models.** For the four examples with shared declarations, the lowered model is written as JSON and compared byte for byte with `expected/effective.json`.
2. **Dependency edges.** For each example, the resolved edges are exported as JSON and compared byte for byte with `expected/dependencies.json`. The test reads only files the pipeline wrote, so it tests the pipeline as a user runs it.
3. **Revisions.** Each `expected/resolved*.json` is the production implementation's resolved document of one application, and records its revision: a digest over the application's complete element. Equal revisions mean that every value of that element matches (Section 4.2.1).
4. **Complete expected models.** Two examples have a complete expected resolved model, compared with EMF's structural equality, `EcoreUtil.equals`, which covers every attribute and reference: `minimal`, written by hand from the specification in Task 1 before the transformation existed, and `secrets`, whose run was reviewed value by value against the specification and the pinned inputs and then committed.
5. **Generated files.** The Acceleo templates of Task 3 decide nothing, so a wrong value in a resolved model becomes a wrong byte in a generated file. Tests generate the files from the transformations' output and compare every file with the committed tree.
6. **Interface tests.** JUnit tests run each transformation on models built in code:
   - the resolution: the smallest project that resolves, a project the input does not contain, and an Asset whose file is missing, which must stop the run with the file's name;
   - the estate pass: one job planned over two projects, a job name that moves when a document changes, no job without documents, and two projects that disagree;
   - the lowering: eight cases of merging shared declarations.

The Java around the transformations must reach full line and branch coverage and a 100% mutation score. Those tools cannot measure QVTo, so the QVTo code is measured only by the comparisons above.

4.2 Results

Table 9 lists the eight examples. All eight resolve, and every comparison that has an expected file passes. The full build, 323 tests including 81 parity cases, passes, and `minimal` and `secrets` equal their complete expected models.

		Counted in the resolved model						Equal to expected files		
Example	**Apps**	**Proc.**	**Edges**	**Secr.**	**Vol.**	**Env.**	**Files**	**Edges**	**Revisions**	**Files**
`minimal`	1	1	0	0	0	6	12	yes	1/1	yes
`auth`	1	2	4	3	0	21	19	yes	1/1	yes
`data`	3	3	0	1	3	3	37	yes	3/3	yes
`delivery`	4	4	1	0	0	0	27	yes	4/4	yes
`edge`	2	2	0	0	0	0	13	yes	–	–
`knowledge-platform`	4	7	5	4	6	84	49	yes	1/1	yes
`observability`	2	2	0	0	0	0	15	yes	–	–
`secrets`	1	1	0	0	0	0	8	yes	–	–

Table 9: The Resolved Examples: Applications, Processes, Dependency Edges, Secrets, Volumes, Environment Variables and Planned Files, and the Comparisons (–: No Expected File)

Together the examples use every row of Table 8. `auth` carries the migration, the placeholders and an edge to a provider outside the cluster; `data` the backups of each durability class, a sidecar and an Asset; `knowledge-platform` is the largest; `delivery` has the rolling and stop-start switchovers and API access; and `edge`, `observability` and `secrets` hold the platform services the others look up.

4.2.1 What the Revisions Show

An application's revision is the digest of its whole element as the production implementation writes it:

- its processes, with image, user, deadline, probes, placement, secrets, volumes, backups, environment and edges;
- its exposures and their routes;
- its release gate and its migration.

The transformation builds that text from its own resolved model (Section 3.7), so one different value, one extra environment variable or one different order changes the digest. Ten applications in five examples have a revision in an expected file, and all ten are reproduced. For these applications the check does the job a hand-written model would, and the expected values come from another implementation. A further test checks that the revision moves for the right reasons: raising `minimal`'s memory from `256Mi` to `512Mi` changes it, while changing the snapshot's capture time, which decides nothing, does not.

4.2.2 What the Generated Files Show

Values no revision covers, such as the path plan, network policies and Vault documents, reach the generated files: 167 committed files of five examples and the cluster-wide tree, the estate pass's job included, are reproduced byte for byte, and twice the same inputs give the same bytes.

4.2.3 Where the Evidence Is Thin

Two examples, `edge` and `observability`, have no revision, no committed tree and no expected model, and their edge files are empty. For them the evidence is that the run succeeds and that the other examples, which look up their proxies and collectors, reproduce their expected files. The `secrets` model is a reviewed output, not an independent oracle: it catches any later change, and the review itself found one gap, in the specification rather than the transformation, since the secret store's network policy admits nobody inbound even though every holder of a secret may send to it. Finally, the hand-written model of `minimal` was updated after Task 1 to follow the metamodel changes of Appendix A.2, and its revision was taken from the production implementation, so that one field no longer checks independently.

4.2.4 Defects Found by Testing

The comparisons found defects one example alone would not show. The clearest: the path plan put the cluster-wide route files under `apps/edge/`, which is also the directory of the project named `edge`. The `edge` example's files would have been mixed with the routes of every other project. The fix moved every cluster-wide file under `estate/`, in both implementations, and the committed trees now hold each file at its new path.

4.3 Limitations

- **The oracles are another implementation's output.** Agreement shows that two implementations of the specification decide the same, not that the specification is right. Only the hand-written model of `minimal` was written from the specification alone.
- **Eight examples are a sample.** They show that every rule runs and agrees on these inputs, not that each rule is right in every case; the QVTo code is not mutation-tested.
- **Checks across documents are stops, not diagnostics.** A stop reports one problem, without its place in the document. Most such checks have moved into the constraints; the remaining stops are a safety net. The cross-document constraints still scan with `allInstances()`; moving them onto one root object is decided but not built.
- **Queries are recomputed.** QVTo keeps mapping results in the trace but caches no query, so `edgeOf` or a surface's address is recomputed wherever it is used, and several queries scan every process of the cluster: the work grows with the square of the number of processes. Parsing, checking, lowering, resolving and writing all eight examples takes about 17 seconds. For a much larger cluster the hot queries would become mappings or precomputed properties, measured first, since mapping calls have their own cost [5].

- **Capacity is checked per machine.** A process is eligible for a machine that can hold it alone; the resolution does not add up what all processes ask of one machine.
- **Two small modelling choices.** When two processes provide the same surface, the first in name order wins; and the order between reconcile units is per project, not per application, because Flux orders whole units.
- **Provenance digests differ by design.** Input digests are hashes over each implementation's own form of the model. Every digest that reaches a generated file, the revision, an Asset's name and the policy job's name, is reproduced.
- **The `knowledge` example is not resolved.** It models a project being retired; its successor, `knowledge-platform`, is.
- **The Eclipse run configuration is not tested.** The build runs the transformations through the standalone executor; the run configuration and the library registration in `plugin.xml` are only run by hand.

4.4 Reflection

The largest single piece of the resolution is not a derivation but the revision: writing the production implementation's JSON form of an application in OCL, key by key, takes about a sixth of `resolution.qvto`, and every mistake in it shows only as a different hash. It was worth it: once the revisions matched, they checked nearly every value of ten applications at once, which a second hand-written model could not have done for the same effort.

QVTo itself fitted the work. Most rules are queries over the whole cluster, which OCL expresses well, and the explicit trace made every reference between created elements visible. Three language details cost time:

- `resolve`, `from` and `access` are keywords, so the black-box unit is not named after its Java package, and two metamodel features are written `_from` and `_access`;
- a multi-statement `if` needs the brace form;
- inside an object expression, `result` still means the enclosing mapping's result, not the object being created.

With hindsight, the evidence would have been planned with the examples. The expected model of `secrets` and the parity checks of `knowledge-platform` were added only at the end, when a review asked which examples nothing compared; and the Task 1 model of `minimal` should have been kept unchanged beside the updated one, so the one independent oracle stays independent.

A Changes to the Task 1 Metamodels

This appendix lists the changes between the metamodels handed in for Task 1 and the metamodels the transformations of Task 2 read and write, found by comparing the two versions of each `.ecore` file class by class and feature by feature. There are four kinds:

- a third metamodel, the pinned inputs (Appendix A.1);
- the target metamodel gained the values the transformation derives and twelve kinds of generated file (Appendix A.2);
- the source metamodel gained the declarations the other examples need (Appendix A.3);
- four grammars (Appendix A.4).

No class was removed. The Task 1 hand-in also showed several OCL helper definitions, such as `levelsAbove` and `derivesDatabase`, as derived attributes in the diagram; they are still OCL definitions in `project-intent.ocl` and were never Ecore features.

A.1 The Pinned Inputs Metamodel

The package `pinnedinputs` holds the documents the resolution reads beside the source models. Task 1 decided that only authored documents get a metamodel; these documents are not authored by a project's owner, and the resolution cannot read them without one, so they get a separate package instead of joining the source metamodel. The node contract is generated from the platform team's description of the machines, another process publishes the images lock and the snapshot, and a project's build writes its migration proof. The source metamodel's structure is also compared with the production implementation's schemas through a committed descriptor, and adding these classes to it would have listed documents those schemas do not hold. No class of the target metamodel refers to them: the resolved model records the values the resolution reads from them, and their digests. Figure 3 shows the metamodel, and Table 10 says what reads each class.

Class	Read by
`NodeContract`, `Node`, `Allocatable`, `Gpu`, `Disk`	`eligible`: memory and CPU as quantities such as `53500m`, and the site, architecture, capabilities, GPUs and disks a process may require.
`ImagesLock`, `LockedImage`	`locked`: every image of a process, sidecar, backup method, migration runner and the Vault policy job.
`ClusterState`, `Binding`	`boundOf`: the machine that already holds a volume.
`CurrentPlacement`, `RollbackBackup`	Not read by the resolution. The snapshot carries them for the delivery process, which the code generation of Task 3 serves.
`MigrationProof`, `ProvenApplication`	`migrationOf` and `gate`: the schema version a release was tested against, and whether it holds a change that cannot run in a transaction.
`AssetFile`	`Asset::resolvedFor`: a configuration file's text. The pipeline creates one per file a project names; it is not parsed.

Table 10: The Classes of the Pinned Inputs Metamodel and the Rules That Read Them

Two attributes are plain strings, an exception to Task 1's rule of enumerations over free-form strings. A node's disk medium is not the source metamodel's `Media` enumeration, because the node contract lists media that no process may request, such as `sdcard`. The resolution compares the spelling instead, and does the same for a node's architecture. The values come from a generated document, not from an author, so the risk the rule guards against, an author's typo, does not arise.

A.2 Changes to the Target Metamodel

The target metamodel has two halves: the decisions the platform takes, and the generated-file entries of the path plan. Both grew.

The root class is unchanged, but what one instance holds changed: Task 1 planned one `ResolvedDeployment` for the whole cluster, and there is now one per project (Section 2.3.2). A `ResolvedApplication` inside it is still the projection one repository publishes.

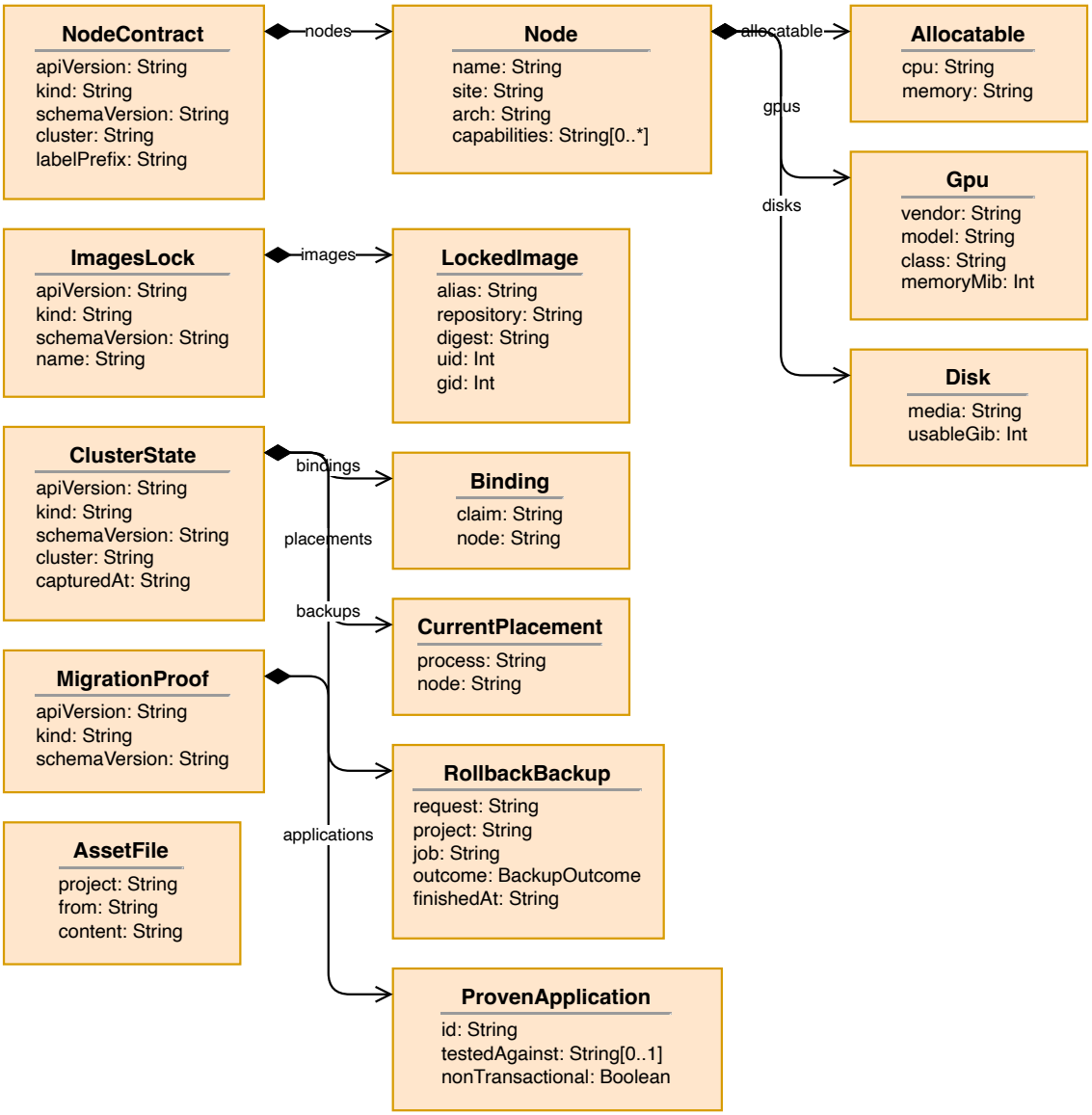


Figure 3: The Pinned Inputs Metamodel. Every Class Is New in Task 2

Decisions. Figure 4 shows every decision class that is new or changed, and Table 11 gives the reason for each change. Task 1 named many of these decisions in prose but gave classes only to what its worked example needed. The other examples also need migrations, backups that connect to their database, secrets that reach the cluster, API access, sidecars, configuration files, and two cluster-wide values: the Vault policy job and the route of the delivery tool's events.

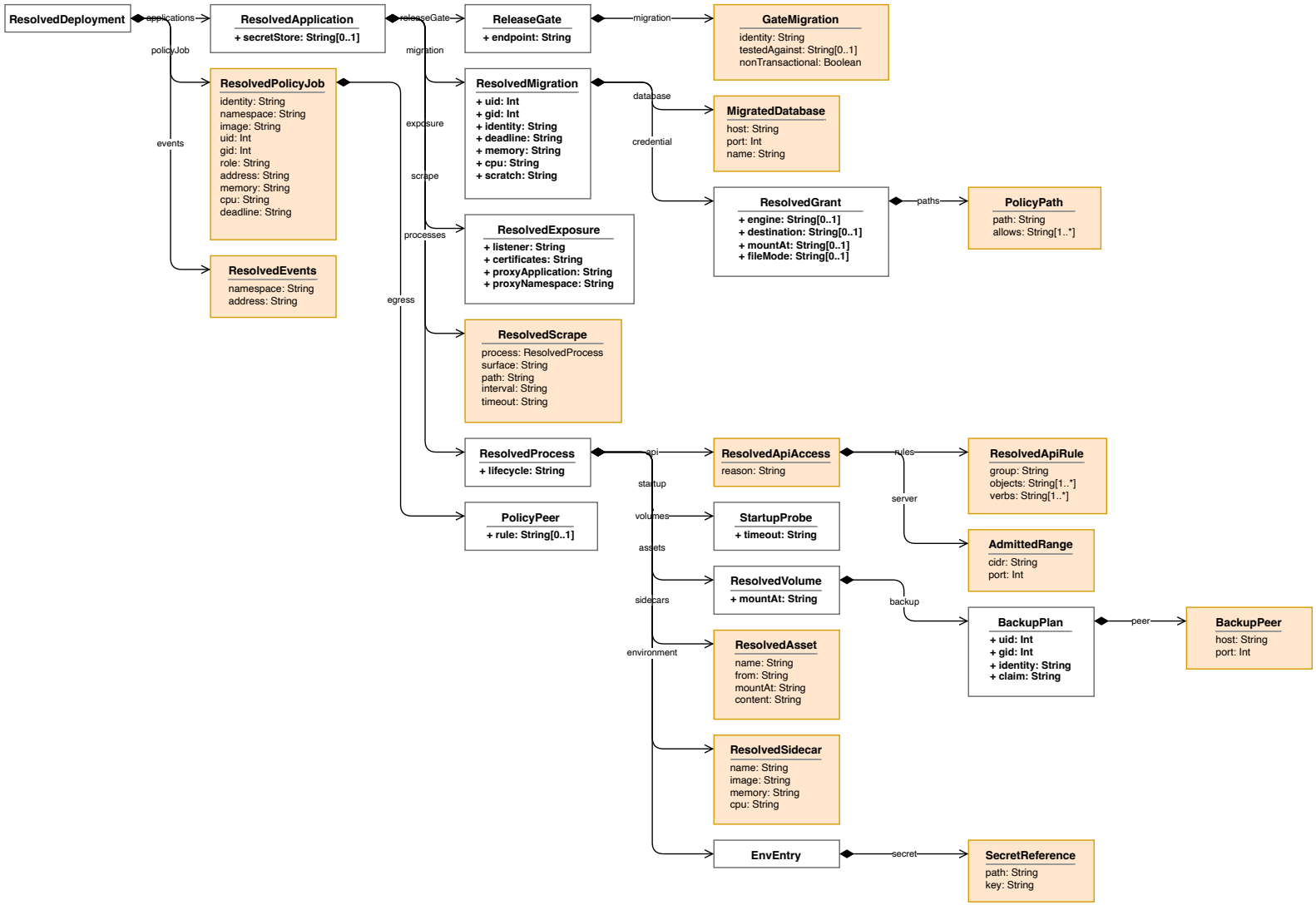


Figure 4: Decision Classes of the Target Metamodel That Changed. Orange: New Class. White: Task 1 Class, Showing Only Its New Features (Bold). Arrows Are Containment; a Class Contained in Several Places Is Drawn Under Its First Container Only

Class	Change	Reason
`ResolvedDeployment`	added `policyJob`, `events`	The Vault policy job and the route of the delivery tool's events belong to the whole cluster; every project's model carries the same one, and the estate pass plans their files.
`ResolvedPolicyJob`, `ResolvedEvents`	new	The job's identity, image, Vault role, resources and egress; the events receiver's namespace and address.
`ResolvedApplication`	added `secretStore`, `scrape`	The secret store's address where the operator syncs from it; how the metrics stack scrapes the application, which the revision's element names.
`ResolvedScrape`	new	The scraped process, surface, path, interval and timeout.
`ReleaseGate`	added `endpoint`, `migration`	The address every Canary's webhooks call, and the migration the gate starts and may undo.
`GateAnalysis`	added `errorRate`, `latency`	The thresholds of the two analysis checks, from the platform.
`GateMigration`	new	The migration's identity and what its proof records.
`ResolvedMigration`	added identity, image user, deadline, memory, CPU, scratch space, `database`, `credential`, `egress`	Task 1 named the identity, owner role and deadline in prose, but the class held only the runner and the proof. A migration Job also needs the address of its database, an owner credential only it holds, and a network policy of its own.
`MigratedDatabase`	new	The database's host, port and name.
`ResolvedProcess`	added `lifecycle`, `api`, `assets`, `sidecars`	Whether the process serves, runs once or prepares, which the revision's element names; its API access, configuration files and sidecars.
`ResolvedApiAccess`, `ResolvedApiRule`, `AdmittedRange`	new	The API rules a process holds, and an address range a policy admits: the API server for API access, a backup's off-cluster destination.
`ResolvedAsset`, `ResolvedSidecar`	new	A configuration file with its content-derived name; a sidecar with its locked image and resources.
`ResolvedGrant`	`path` and `access` optional; added `engine`, `paths`, `destination`, `mountAt`, `fileMode`	A transit or database secret has no single path; it is recorded as the Vault paths its policy covers. A synced secret needs the Kubernetes Secret it is copied to, and a file secret its mount.
`PolicyPath`	new	One path of a secret's policy and the operations allowed on it.
`ResolvedVolume`	added `mountAt`	The workload template needs the mount path.
`BackupPlan`	added user, group, identity, claim, `egress`, `peer`, `credential`, `destinations`	A backup runs as its own identity with its own claim and network policy, connects to the process it dumps, and copies off the cluster with a credential.
`BackupPeer`	new	Where a backup connects, and the credential it logs in with.
`EnvEntry`	`value` optional; added `secret`	A secret placeholder becomes a reference, so the value never appears in a file.

Class	Change	Reason
`SecretReference`	new	The secret path and key a variable reads.
`StartupProbe`	added `timeout`	The specification gives the startup probe the platform's probe timeout.
`ResolvedExposure`	added `listener`, `certificates`, `proxyApplication`, `proxyNamespace`	The entry point's protocol and certificate source, and the proxy whose middleware the routes name.
`PolicyPeer`	added `rule`; `process` optional	Which rule admitted the peer, so two rules that admit one peer stay two entries, as in the production implementation; a peer such as the cluster's DNS is a namespace without a process.

Table 11: Changes to the Decision Classes of the Target Metamodel

Generated Files. Figure 5 shows every subclass of `Deliverable`. Task 1 had nine; there are now 21. Three kinds of change were made.

- **One file per application.** The workload, Canary and identity files, which Task 1 tied to one process, now list `processes`, because the specification writes one file per application holding the resources of all its processes; Task 1's worked example had one process, so the difference did not show. For the same reason the network policy file holds one `ProcessPolicy` per process, and the secret file refers to its application and holds one `SecretHolder` per identity, since a backup identity holds secrets too.
- **Monitors.** `MonitorFile` gained `timeout` and a `kind`: a blue/green process is scraped through its pods, any other through its service.
- **Twelve new kinds of file**: services, volume claims, configuration maps, backups, disruption budgets, migrations, RBAC, the secret operator's connection, Vault policies and roles, the policy job and the events route.

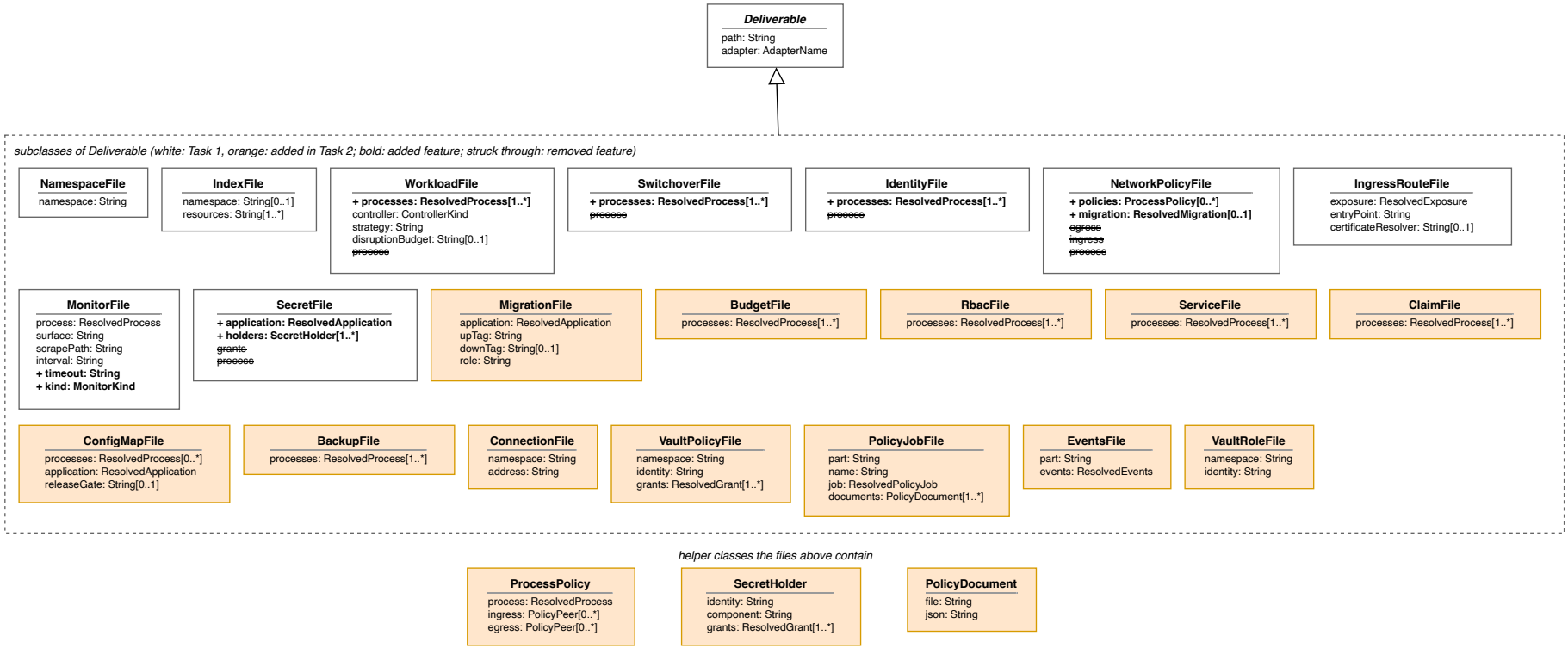


Figure 5: The Generated-File Classes of the Target Metamodel, Task 1 Against Task 2

The hand-written model of `minimal` from Task 1 was updated with these changes: its files refer to the process through `processes`, it gained the configuration map that carries the release gate's inputs, and its revision was set to the production implementation's (Section 4.2.3).

A.3 Changes to the Source Metamodel

The source metamodel kept every class. Table 12 lists what was added or changed. Most additions are declarations in the platform document that a derivation needs and that Task 1 had left implicit; a rule that needs a value the platform does not declare would otherwise have to contain it as a constant.

Class	Change	Reason
`Process`	added `api` (`ApiAccess`, `ApiRule`, `ApiVerb`)	A process declares the Kubernetes API access it needs, with a reason; the platform's delivery tools need it.
`Platform`	added `apiAccess` (`ApiAccessPolicy`)	Which applications may hold API access, and the API server's address range.
`Platform`	added `secretStore`, `telemetry` (`TelemetryPolicy`), `policyJob` (`VaultPolicyJob`)	References to the secret store, the telemetry collector and the metrics stack, whose addresses the resolution finds in their projects; and the Vault policy job's image, role and resources.
`DeliveryPolicy`	added `gate`, `reporter`	References to the release gate and the events receiver applications.
`AnalysisPolicy`	added `errorRate`, `latency`	The thresholds the gate checks.
`Substrate`	added `clusterDns`	The namespace every egress policy admits for DNS.
`EnginePolicy`	added `surface`	The port a database engine's backup connects to.
`OffClusterCopy`	added `egress` (`DestinationRange`)	The address ranges a backup's copy goes to.
`HandoverLedger`	added `serving` (`ServingLocation`)	Where a project that has not moved yet still runs, so an edge to it resolves.
`RenderedArtifacts`	added `pullSecret`	Used by delivery, not by the resolution.
`Placeholder`	added `suffix`	Text after a placeholder, as in `${exposure:auth.public#url}/login`.
`Grant`	`rotation` required	Every grant states how it tolerates rotation, which decides its restart targets.
`Volume`	`size` required	A claim cannot be generated without a size.
`EffectiveProject`	removed `apiVersion`, `kind`, `schemaVersion`	The lowered model is written by the lowering, not by an author, so it has no document header.

Table 12: Changes to the Source Metamodel

A.4 Grammars for the Pinned Inputs

The pinned inputs are read with grammars rather than a Java YAML reader or hand-written XMI, so that both implementations read the same YAML files and their results can be compared. Four grammars were added beside the project and platform grammars: `NodeContract`, `ImagesLock`, `ClusterState` and `MigrationProof`. Each inherits the project grammar, so it reuses its terminals, its indentation tokens and its scalar rule, and adds only the rules for its own document. As with the platform document, the file name selects the grammar: `node-contract.yml`, `images.lock.yml`, `cluster-state.yml` and `migration-proof.yml`. Example 4 shows the smallest. One input file changed: the project grammar's scalar terminal cannot contain a colon, so the snapshot's capture time is now quoted, as digests and URLs already were. The production implementation reads the same value either way.

```
grammar dev.jorisjonkers.deploykit.emf.syntax.ClusterState
```

```
        with dev.jorisjonkers.deploykit.emf.syntax.ProjectIntent
import "https://jorisjonkers.dev/deploy-kit/pinned-inputs/1"

ClusterState returns ClusterState:
        (('apiVersion' ':' apiVersion=Text)
        & ('kind' ':' kind=Text)
        & ('schemaVersion' ':' schemaVersion=Text)
        & ('cluster' ':' cluster=Text)
        & ('capturedAt' ':' capturedAt=Text)
        & ('bindings' ':' BEGIN (DASH BEGIN bindings+=Binding END)* END)
        & ('placements' ':' BEGIN (DASH BEGIN placements+=CurrentPlacement END)* END)
        & ('backups' ':' BEGIN (DASH BEGIN backups+=RollbackBackup END)* END)?);

Binding returns Binding:
        (('claim' ':' claim=Text)
        & ('node' ':' node=Text));
```

Example 4: The Grammar of the Cluster-State Snapshot, Abridged

B The Hand-in Archive

The archive holds this report as `report.pdf`, a `README.md` with the steps of Appendix B.1, and two folders:

- `task-2-transformations/`, an Eclipse project for reading the models (Table 13), imported with File > Import > Existing Projects into Workspace. It contains no Java, so the transformations do not run from it: the resolution and the estate pass call the `Hashing` library, which the implementation's plug-in registers;
- `implementation/`, the EMF implementation and the examples its tests read, from which the transformations run.

Path	Contents
`model/`	The source, pinned-inputs and target metamodels.
`transforms/`	`lower.qvto`, `resolution.qvto` and `estate.qvto`.
`examples/platform/`	The platform document and the pinned inputs every example reads.
`examples/<example>/authored/`	The project document with its env and configuration files.
`examples/<example>/resolution.intent.xmi`	The intent extent: authored and lowered projects, and the platform document.
`examples/<example>/resolution.pinned.xmi`	The pinned extent.
`examples/<example>/<project>.resolveddeployment`	The resolved model the transformation wrote.
`examples/<example>/dependencies.json`	The edges exported from it, beside `expected-dependencies.json`, a copy of the oracle `expected/dependencies.json`.
`examples/minimal/minimal.hand-written.resolveddeployment`	The hand-written target model of Task 1, as changed in Appendix A.2.
`examples/secrets/secrets.expected.resolveddeployment`	The complete expected model of `secrets` (Section 4.2.3).

Table 13: The Models Project of the Archive, One Folder per Example of Table 9

Every XMI declares its metamodel in `xsi:schemaLocation`, so Eclipse opens it in the Sample Reflective Ecore Model Editor without a registered package.

B.1 Running the Transformations

1. In `implementation/emf`, run `./mvnw clean verify` with JDK 21. It builds the implementation, runs every test, and writes each example's models and edges under `bundles/cli/target/parity/<example>/`.
2. In Eclipse Modeling Tools with the QVT-Operational SDK, import `implementation/emf` as existing Maven projects.
3. Run `resolution.launch` in the project `dev.jorisjonkers.deploykit.emf.resolve`. It resolves `minimal` from the extents step 1 wrote.

References

[1] P. Merle and F. Petrillo. Visualizing Cloud-native Applications with KubeDiagrams. In *Proc. VISSOFT 2025*, pp. 106–116, 2025. Tool: https://github.com/philippemerle/KubeDiagrams.

[2] KubeOrch, visual Kubernetes workflow orchestrator. https://github.com/KubeOrch (repositories `core` and `ui`), visited 15 October 2026.

[3] kubert, a kubectx and kubens alternative. https://github.com/idebeijer/kubert, visited 15 October 2026.

[4] L. Burgueño, J. Cabot and S. Gérard. The Future of Model Transformation Languages: An Open Community Discussion. *Journal of Object Technology* 18(3), 7:1–11, 2019.

[5] M. van Amstel, S. Bosems, I. Kurtev and L. Ferreira Pires. Performance in Model Transformations: Experiments with ATL and QVT. In *Proc. ICMT 2011*, LNCS 6707, pp. 198–212, 2011.

[6] Object Management Group. *Meta Object Facility Query/View/Transformation Specification*, version 1.3, 2016.